



# **REGRESSÃO LINEAR MÚLTIPLA**

## **APRENDIZAGEM DE MÁQUINA**

### **CST em Desenvolvimento de Software Multiplataforma**



**PROF. Me. TIAGO A. SILVA**



# O QUE É REGRESSÃO LINEAR MÚLTIPLA?

- A Regressão Linear Múltipla é um dos pilares da estatística e do aprendizado de máquina supervisionado para problemas de previsão contínua.
  - Expande o conceito da regressão linear simples ao **utilizar duas ou mais variáveis independentes (features)** para modelar e prever o valor de uma única variável dependente (**target**).
- Visualmente, enquanto a regressão simples traça uma reta em um gráfico 2D, a regressão múltipla com duas variáveis independentes ajusta um plano em um espaço 3D para minimizar a distância entre os pontos de dados reais e as previsões. Com três ou mais variáveis, **o modelo ajusta um hiperplano em um espaço multidimensional**.

# COMO FUNCIONA REGRESSÃO LINEAR MÚLTIPLA?

- Na regressão simples, você tem algo assim:

$$y = \beta_0 + \beta_1 x$$

- Na regressão múltipla, isso se expande para:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

- Vamos colocar isso em um formato mais formal:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

# COMO FUNCIONA REGRESSÃO LINEAR MÚLTIPLA?

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

- O que cada termo significa:
  - $y$  é a variável dependente (o valor que queremos prever).
  - $\beta_0$  é o intercepto (o valor esperado de  $y$  quando todas as variáveis  $x$  são iguais a zero).
  - $\beta_1 + \beta_2 + \dots + \beta_n$  são os coeficientes de regressão (ou **pesos**). Cada  $\beta_i$  representa a mudança média em  $y$  para cada aumento de uma unidade em  $x_i$ , assumindo que todas as outras variáveis permaneçam estritamente constantes.
  - $x_1 + x_2 + \dots + x_n$  são As variáveis independentes (nossas **features** ou **dados de entrada**).
  - $\epsilon$  é o termo de erro (ou resíduo), que captura a variação em  $y$  que não consegue ser explicada pelas variáveis do modelo.

## UM EXEMPLO MAIS PRÁTICO...

- Imagine que você quer prever o preço de uma casa:

$$preço = \beta_0 + \beta_1 \cdot \underset{\uparrow}{\text{área}} + \beta_2 \cdot \underset{\uparrow}{\text{quartos}} + \beta_3 \cdot \underset{\uparrow}{\text{idade}}$$

- Onde:
  - *área* influencia o preço
  - Número de *quartos* também
  - *idade* do imóvel também
- Cada variável contribui com um “peso”:  $\beta_i$ .**

# COMO O MODELO APRENDE OS PESOS?

- O objetivo é encontrar os valores de  $\beta_i$  (betas) que minimizam o erro entre:
  - valores reais ( $y$ )
  - valores previstos ( $\hat{y}$ )
- Isso normalmente é feito minimizando o **erro quadrático**:

$$Erro = \sum (y - \hat{y})^2$$

# COMO O MODELO APRENDE OS PESOS?

- Para implementar com NumPy, usamos a forma vetorial:

$$\hat{y} = X \cdot \beta$$

- E a solução clássica (equação normal) é:

$$\beta = (X^T X)^{-1} X^T y$$

# COMO O MODELO APRENDE OS PESOS?

- E a solução clássica (equação normal) é:  $\beta = (X^T X)^{-1} X^T y$

| área | quartos | preço |
|------|---------|-------|
| 50   | 2       | 200   |
| 80   | 3       | 300   |
| 100  | 4       | 400   |

$$X = \begin{bmatrix} 1 & 50 & 2 \\ 1 & 80 & 3 \\ 1 & 100 & 4 \end{bmatrix}$$

$$y = \begin{bmatrix} 200 \\ 300 \\ 400 \end{bmatrix}$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

- O objetivo é calcular:  $X \cdot \beta = y$

# COMO O MODELO APRENDE OS PESOS?

- Mas nem sempre é possível resolver assim  $X \cdot \beta = y$ , porque:
  - Geralmente há mais linhas do que colunas e o sistema é aproximado.
  - Então o truque é usar os mínimos quadrados:  $\beta = (X^T X)^{-1} X^T y$
  - Que faz o seguinte:
    - 1) Transposta  $X^T$  que troca linhas por colunas, para “alinhar” as dimensões.
    - 2) Multiplica  $X^T X$  gerando uma matriz quadrada.
    - 3) Inverte  $(X^T X)^{-1}$  para desfazer a influencia sobre as variáveis.
    - 4) Multiplica  $X^T y$  para incorporar os valores reais

# COMO O MODELO APRENDE OS PESOS?

- Fazendo uma analogia:
  - $X$  são os dados de entrada (informação bruta, as features)
  - $y$  é o resultado real (não confundir com o  $y$  chapéu)
  - A formula então encontra os pesos beta que melhor explicam  $y$  a partir das features  $X$
  - Como se fizéssemos “na mão”:  $preço = a + b \cdot área + c \cdot quartos$
  - Essa fórmula projeta os dados em um espaço onde:
    - O erro entre previsão e realidade é mínimo
    - Estamos encontrando o “melhor ajuste possível”

# CALCULANDO “NA MÃO”

| área | quartos | preço |
|------|---------|-------|
| 50   | 2       | 200   |
| 80   | 3       | 300   |
| 100  | 4       | 400   |

$$X = \begin{bmatrix} 1 & 50 & 2 \\ 1 & 80 & 3 \\ 1 & 100 & 4 \end{bmatrix} \quad y = \begin{bmatrix} 200 \\ 300 \\ 400 \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$$\beta = (X^T X)^{-1} X^T y$$

# CALCULANDO “NA MÃO”: TRANSPORTA DA MATRIZ $X$ : $X^T$

| área | quartos | preço |
|------|---------|-------|
| 50   | 2       | 200   |
| 80   | 3       | 300   |
| 100  | 4       | 400   |

$$X = \begin{bmatrix} 1 & 50 & 2 \\ 1 & 80 & 3 \\ 1 & 100 & 4 \end{bmatrix} \quad \rightarrow \quad X^T = \begin{bmatrix} 1 & 1 & 1 \\ 50 & 80 & 100 \\ 2 & 3 & 4 \end{bmatrix}$$

# CALCULANDO “NA MÃO”: CALCULAR $X^T X$

- Linha 1:
  - $(1,1): 1 + 1 + 1 = 3$
  - $(1,2): 50 + 80 + 100 = 230$
  - $(1,3): 2 + 3 + 4 = 9$
- Linha 2
  - $(2,1): 230$
  - $(2,2): 50^2 + 80^2 + 100^2 = 2500 + 6400 + 10000 = 18900$
  - $(2,3): 50 \cdot 2 + 80 \cdot 3 + 100 \cdot 4 = 100 + 240 + 400 = 740$
- Linha 3
  - $(3,1): 9$
  - $(3,2): 740$
  - $(3,3): 2^2 + 3^2 + 4^2 = 4 + 9 + 16 = 29$

## CALCULANDO “NA MÃO”: CALCULAR $X^T X$

$$X^T X = \begin{bmatrix} 3 & 230 & 9 \\ 230 & 18900 & 740 \\ 9 & 740 & 29 \end{bmatrix}$$

## CALCULANDO “NA MÃO”: CALCULAR $X^T y$

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 50 & 80 & 100 \\ 2 & 3 & 4 \end{bmatrix} \quad . \quad y = \begin{bmatrix} 200 \\ 300 \\ 400 \end{bmatrix}$$



$$X^T y = \begin{bmatrix} 200 + 300 + 400 \\ 50 \cdot 200 + 80 \cdot 300 + 100 \cdot 400 \\ 2 \cdot 200 + 3 \cdot 300 + 4 \cdot 400 \end{bmatrix}$$



$$X^T y = \begin{bmatrix} 900 \\ 74000 \\ 2900 \end{bmatrix}$$

# COMO FICARIA EM PYTHON?

```
1 import numpy as np
2
3 # matriz X (com coluna de 1s)
4 X = np.array([
5     [1, 50, 2],
6     [1, 80, 3],
7     [1, 100, 4]
8 ])
9
10 # vetor y
11 y = np.array([200, 300, 400])
12
13 # fórmula da regressão múltipla
14 beta = np.linalg.inv(X.T @ X) @ X.T @ y
15
16 print(beta)
```

- **X.T** calcula a matriz transposta de X ( $X^T$ )
- **@** é o operador do Python para multiplicação de matrizes.
- **np.linalg.inv(...)** calcula a matriz inversa do que está entre parênteses ( $(X^T X)^{-1}$ ).
- Tudo isso é multiplicado novamente por  $X^T$  e depois por  $y$  para encontrar o vetor  $\beta$ .
- Resultado é um “vetor de betas”.

# COMO O MODELO APRENDE OS PESOS?

- E a solução clássica (equação normal) é:  $\beta = (X^T X)^{-1} X^T y$

| área $x_1$ | quartos $x_2$ | preço $y$ |
|------------|---------------|-----------|
| 50         | 2             | 200       |
| 80         | 3             | 300       |
| 100        | 4             | 400       |

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$$preço = \beta_0 + \beta_1 \cdot área + \beta_2 \cdot quartos$$

# IMPLEMENTAÇÃO NO JUPYTER NOTEBOOK



# OBRIGADO!

- Encontre este **material on-line** em:
  - Slides: **Google Classroom**
- Em caso de **dúvidas**, entre em contato:
  - **Prof. Me. Tiago:** [tiago.silva@cps.sp.gov.br](mailto:tiago.silva@cps.sp.gov.br)

